

# THE LOVE FOR RECURSION

---

The Java stack, function calls and recursion.  
A recurring dilemma.

**Florian Sihler**

September 21, 2026  
SP, Ulm University

# THE LOVE FOR RECURSION

---

The Java stack, function calls and recursion.  
A recurring dilemma.

Florian Sihler



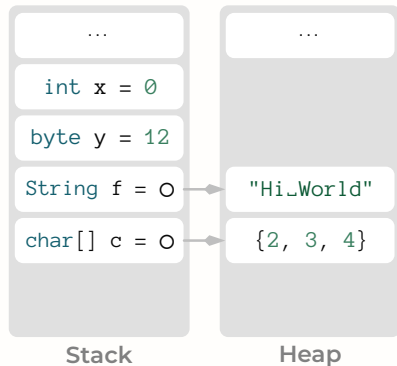
Officially supported by the Pingu-Foundation for Emotional Support. There will be light and there will be more peeps to come!

September 21, 2026  
SP, Ulm University

## 2

# Java's Memory Management

```
int x = 0;  
byte y = 12;  
String f = "Hi_World";  
char[] c = {2, 3, 4};
```



[5]: The Java® Virtual Machine  
Specification [Heap & Stack]  
Lindholm, 2020



Note: in Java every variable gets a “default value”. For complex data types this is **null**.

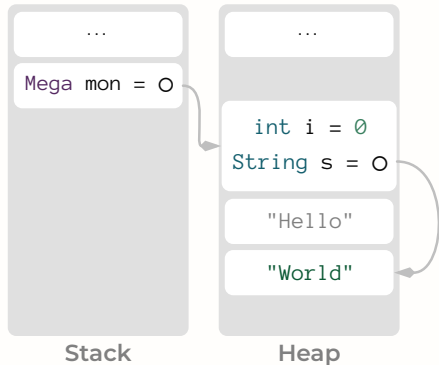


### 3

## More Complex Constructs

```
class Mega {  
    public int i;  
    public String s;  
}
```

```
Mega mon = new Mega();  
mon.s = "Hello";  
mon.s = "World";
```



[5]: The Java® Virtual Machine  
Specification [Heap & Stack]  
Lindholm, 2020

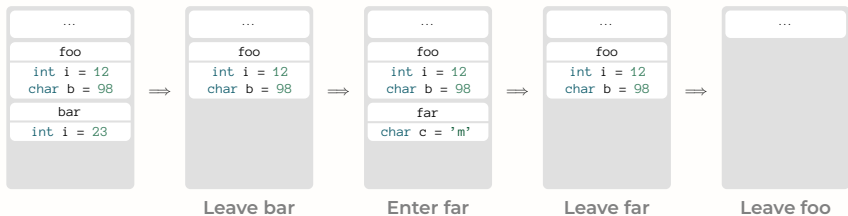
[8]: EPK-Package, tikzpingus  
Sihler, 2021



## 4

# Stack Frames

- Handle (local) scopes.
- Are pushed onto the stack when entered.
- Are taken off the stack again when left.



[3]: Java Debug Interface –  
StackFrame (JDK 11)

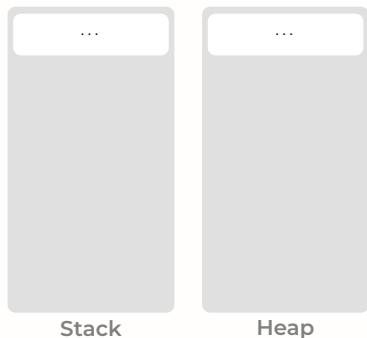
, 2021



## 5.1

# Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```

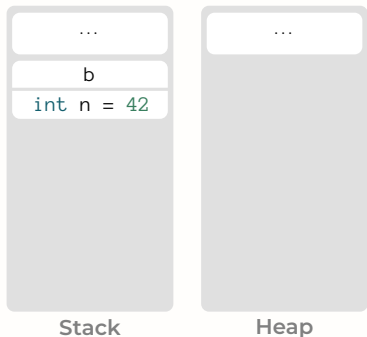




## 5.2

# Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
• String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```

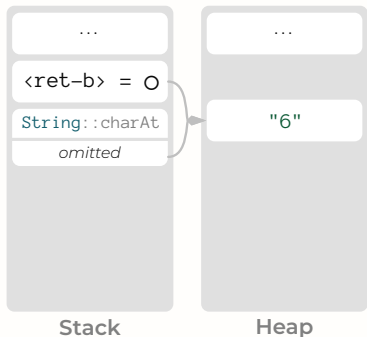




## 5.3

# Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```



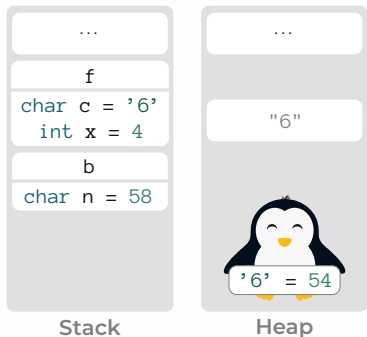




## 5.4

# Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
• String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```

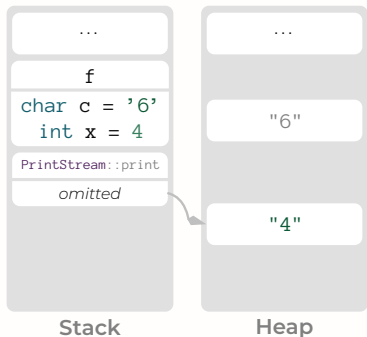




## 5.5

# Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    • System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```



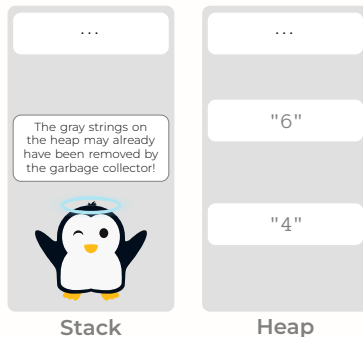
## 5.6

# Stack Frames – Example

We are going to simplify return mechanisms shamefully!



```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0)); •
```



[3]: Java Debug Interface –  
StackFrame (JDK 11)

, 2021



## 6

# Anatomy of a Recursion

```
1 func recursive(problem arguments) is  
2   | if at elemental problem then  
3   |   solve elemental problem;  
4   | else  
5   |   Split problem into smaller ones;  
6   |   Try to solve them recursively;  
7   |   Combine step-solutions to one;  
8   | end  
9 end
```



Not all of them  
have to follow ex-  
actly this structure.  
Be special. Be wild!

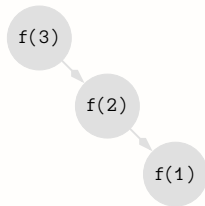
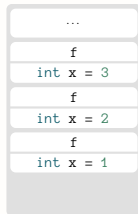


# 7

## Stack Frames and Call Graphs

- Stack frames quickly get confusing.
- We simplify this with call graphs.
- Nodes represent stack frames.

```
int f(int x) {  
    return f(x - 1);  
}
```



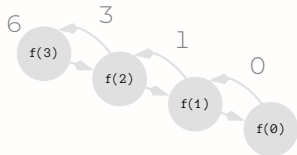
## 8

## Linear Recursion

Linear recursion is subdivided further! For example into head and tail recursion.



```
int f(int n) {  
    if (n <= 0) return n;  
    else return n + f(n - 1);  
}
```



The simplest and most common form of recursion.

This one is even a *primitive recursive* function!<sup>[2]</sup>

[2]: About primitive recursive algorithms  
Colson, 1991



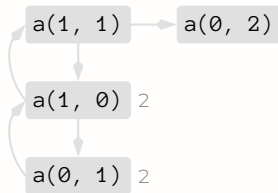
## 9.1

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {  
    if(m == 0) return n + 1;  
    else if (n == 0) return a(m - 1, 1);  
    else return a(m - 1, a(m, n - 1));  
}
```



[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

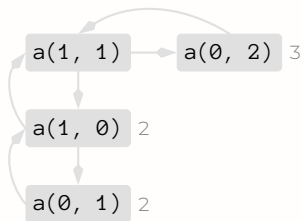
## 9.2

## Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {  
    if(m == 0) return n + 1;  
    else if (n == 0) return a(m - 1, 1);  
    else return a(m - 1, a(m, n - 1));  
}
```



Nested recursive functions  
are usually very hard to follow.



Hence  $a(1, 1) = 3$ .

So what is, for example,  $a(4, 2)$ ?

Well...  $a(4, 2) \approx 2 \cdot 10^{19729}$ .

[9]: The Ackermann function, a  
theoretical, computational, and  
formula manipulative study  
Sundblad, 1971

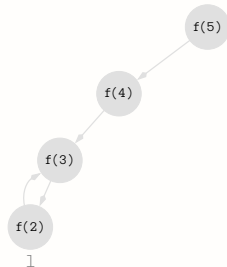




## 10.1

# Cascading Recursion

```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



It is sometimes also called  
"branching recursion".

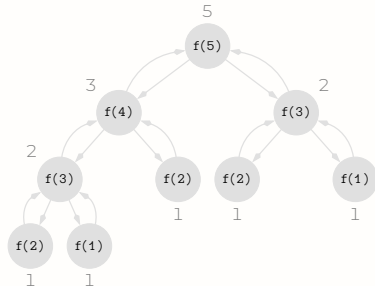


## 10.2

# Cascading Recursion

```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```

Elegant, but usually not  
particularly efficient.



[7]: Exploring Recursion with  
Fibonacci Numbers  
Rubio-Sánchez, 2007

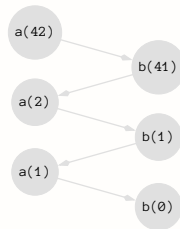




## 11.1

# Mutual Recursion

```
int a(int x) {  
    if(x <= 0) return x + 1;  
    return b(x - 1);  
}  
  
int b(int x) {  
    if(x <= 0) return x;  
    return a(x % 3);  
}
```



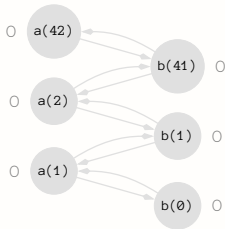
With function tuples it can  
always be transformed  
into “normal” recursion.



## 11.2

# Mutual Recursion

```
int a(int x) {  
    if(x <= 0) return x + 1;  
    return b(x - 1);  
}  
  
int b(int x) {  
    if(x <= 0) return x;  
    return a(x % 3);  
}
```



# 12

## A Few Notes

- The examples shown were kept as minimal as possible
- The variants can be combined!
  - › e.g. a cascading, mutual recursion.
- Recursion and iteration are usually equally powerful.

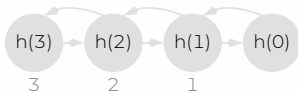
# 13

## Head vs. Tail Recursion

In short: does something still happen on the way back up the recursion or not (tail).

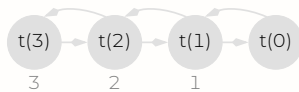


```
void h(int n) {  
    if(n <= 0) return;  
    h(n - 1);  
    System.out.println(n);  
}
```



Output: 1 → 2 → 3

```
void t(int n) {  
    if(n <= 0) return;  
    System.out.println(n);  
    t(n - 1);  
}
```



Output: 3 → 2 → 1

Disclaimer: the subtleties of head recursion are neglected here. Tail recursion is more interesting because it can be transformed into iteration without any trouble.



## 14

# Mathematical Functions

- Some mathematical functions are defined recursively.
- The given definition can usually be translated 1:1.
- It already takes a good deal of the thinking off your hands.

"Elemental problems" are the non-recursive cases here: parts of the sequence that are often given axiomatically.



$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

$$f_n : \mathbb{N} \rightarrow \mathbb{N}, n \mapsto n \cdot f_{n-1}$$
$$f_1 = 1$$

```
public int f(int n) {  
    if(n <= 1) return 1;  
    else return n * f(n - 1);  
}
```

```
int f(int n) {  
    return phi(n, n, n);  
}  
  
int alpha(int a, int n) {  
    return n <= 1 ? n : a;  
}
```



$$f(n) = \phi(n, n, n)$$

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$

$$\alpha(a, n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ a, & \text{if } n > 1 \end{cases}$$



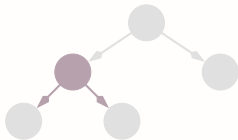
```
int phi(int a, int b, int n) {  
    if(n == 0) return a + b;  
    else if (b == 0) return alpha(a, n - 1);  
    else return phi(a, phi(a, b - 1, n), n - 1);  
}
```

$f(1) = 1, f(2) = 4, f(3) = \text{⚡}$

$\gtrsim 2 \uparrow^2 7 \approx 10^{10^{10^{19000}}}$

# 17

## Recursive Structures



Every node is the root of a subtree

```
class Node {  
    Node left;  
    Node right;  
    int data;  
}
```

```
class Node<T> {  
    Node left;  
    Node right;  
    T data;  
}
```

Cooler with generics!<sup>[4]</sup>



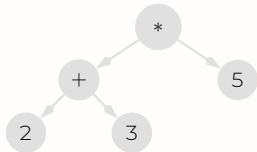
Every element is the head of a sublist

```
class Element {  
    Element next;  
    int data;  
}
```



## 18.1

# Evaluating a Tree



An arithmetic tree!

```
class Node {  
    Node left, right;  
    int data; String op;  
}
```

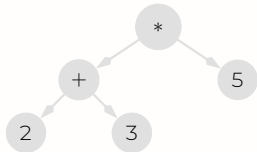
```
long postOrderEval(Node node) {  
    if(node == null) throw new /* ... */;  
    if(node.op == null) // no operation  
        return node.data;  
    long l = postOrderEval(node.left);  
    long r = postOrderEval(node.right);  
    switch(node.op) {  
        case "+": return l + r;  
        case "*": return l * r;  
        // ... other cases, errors, ...  
    }  
}
```

In practice one would realise the nodes differently. E.g. through inheritance.



## 18.2

# Evaluating a Tree



An arithmetic tree!

```
class Node {  
    Node left, right;  
    int data; String op;  
}
```

```
long postOrderEval(Node node) {  
    if(node == null) throw new /* ... */;  
    if(node.op == null) // no operation  
        return node.data;  
    long l = postOrderEval(node.left);  
    long r = postOrderEval(node.right);  
    switch(node.op) {  
        case "+": return l + r;  
        case "*": return l * r;  
        // ... other cases, errors, ...  
    }  
}
```

In practice one would re-  
alise the nodes differently.  
E.g. through inheritance.



# 19

## The World of Recursive Problems

### 1. Recursive sorting

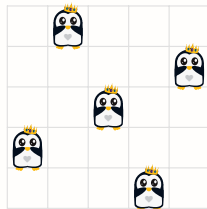


- Sudokus
- Pathfinding
- Hilbert-Kurven
- ...

### 2. The towers of Hanoi



### 3. The N-queens problem



Everything that can be split into (possibly explorative) single steps.



```
void quicksort(int[] arr) {  
    quicksort(arr, 0, arr.length - 1);  
}
```

```
void quicksort(int[] arr, int left, int right) {  
    if(low >= right) return;  
    int partIndex = partition(arr, left, right);  
    quicksort(arr, left, partIndex - 1);  
    quicksort(arr, partIndex + 1, right);  
}
```



Picks a pivot and sorts everything smaller than the pivot to the left and everything greater or equal to the right.



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

```
enum Pole {
    SOURCE,
    HELPER,
    TARGET
}
```

```
void hanoi(int disks, Pole a, Pole b, Pole c) {
    if(disks == 1) { move(a, c); }
    else {
        hanoi(disks - 1, a, c, b); // move all disks above to help pole
        move(a, c); // move last disk to target
        hanoi(disks - 1, b, a, c); // move disks from help to target pole
    }
}
```

In the simplest case it just prints " $a \rightarrow c$ ".



## 22

## The N-Queens Problem

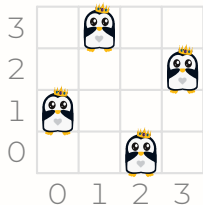
```
boolean nQueens(boolean[][] board) {
    return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
    if(row == board.length) return true;
    for (int column = 0; column < board.length; column++) {
        if(!queenCanBePlacedHere(board, column, row)) continue;
        board[row][column] = true; // Try to place queen here
        if(nQueens(board, row + 1))
            return true;
        board[row][column] = false;
    }
    return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



q(board, 0)



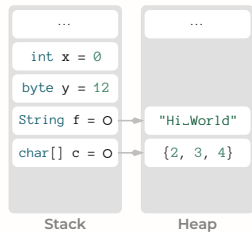


## The Obligatory “By the Way...”

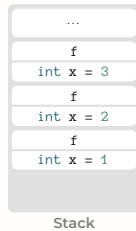
- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!
- The huge topic of compiler construction offers more:
  - Optimizations & transformations
  - Recursive Descent Parser
- Dynamic programming!



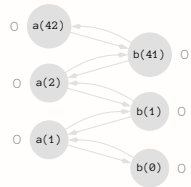
## 24.1



Memory  
management



Stack Frames

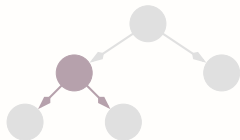


Recursion types

## 24.2

$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

Mathematical



Data structures



Divide- & Conquer



- [1] William D. Clinger. “Proper Tail Recursion and Space Efficiency”. 1998
- [2] Loïc Colson. “About primitive recursive algorithms”. 1991
- [3] *Java Debug Interface – StackFrame (JDK 11)*. 2021
- [4] *Lesson: Generics*. 2021
- [5] Tim Lindholm et al. *The Java® Virtual Machine Specification [Heap & Stack]*. 2020
- [6] Yanhong A. Liu and Scott D. Stoller. “From Recursion to Iteration: What Are the Optimizations?” Nov. 1999
- [7] Manuel Rubio-Sánchez and Isidoro Hernán-Losada. “Exploring Recursion with Fibonacci Numbers”. 2007
- [8] Florian Sihler. *L<sup>A</sup>T<sub>E</sub>X-Package, tikzpingus*. 2021
- [9] Yngve Sundblad. “The Ackermann function. a theoretical, computational, and formula manipulative study”. 1971



**Florian Sihler**

Ulm, September 21, 2026